30/12

Week - 4

Cuckoo Search Algo for:
Robot Path Planning.

1. Generate 'N' random paths from start to Target
2. Evaluate the length/safety of all 'N' paths.
3. while (Not finished / Max rounds not reached):
4. Pick a random path 'A'
5. Create a new path 'B' by slightly 'A' using Levy Flight jump
6. Evaluate path 'B'
7. Pick random path 'C' from the list
8. If (Path 'B' is shorter / safer than path 'C'):
9. Replace path 'C' with path 'B'
10. Identify the worst paths in the list (Probability Pa)

Bafna Gold
Date: Page:

Print

11. Delete those worst paths & replace them with brand new random paths
12. Remember the best path found so far
13. End while
14. Return the best collision-free path

o/p. Optimal Waypoint found: X = 7.12,
Y = 7.12
Total distance: 14.14 units

o/p

30/10/25

```python
import random
import math

# Input
sx, sy = map(float, input("Enter start point: ").split())
tx, ty = map(float, input("Enter target point: ").split())

n = int(input("Enter number of nests: "))
rounds = int(input("Enter maximum rounds: "))

# Generate random paths
def create_path():
    return [
        (random.uniform(min(sx, tx), max(sx, tx)),
         random.uniform(min(sy, ty), max(sy, ty)))
        for _ in range(3)
    ]

# Calculate distance
def distance(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

# Evaluate path length
def fitness(path):
```

```python
# Evaluate path length
def fitness(path):
    points = [(sx, sy)] + path + [(tx, ty)]
    total = 0

    for i in range(len(points)-1):
        total += distance(points[i], points[i+1])

    return total

# Levy flight
def levy_flight(path):
    new_path = []

    for x, y in path:
        x += random.gauss(0, 1) * random.uniform(-2, 2)
        y += random.gauss(0, 1) * random.uniform(-2, 2)
        new_path.append((x, y))

    return new_path

# Initialize nests
nests = [create_path() for _ in range(n)]

best_path = min(nests, key=fitness)
```

```
# Cuckoo Search
for iteration in range(rounds):

    # Pick random path A
    a = random.randrange(nests.__len__())

    # Create new path B using Levy flight
    new_path = levy_flight(nests[a])

    # Pick random path C
    c = random.randrange(n)

    # Replace if new path is better
    if fitness(new_path) < fitness(nests[c]):
        nests[c] = new_path

    # Identify and remove worst paths
    nests.sort(key=fitness)

    pa = 0.25
    remove = int(pa * n)

    for i in range(remove):
        nests[-(i+1)] = create_path()
```

```
        print("Iteration:", iteration + 1,
              "Best distance:", round(fitness(best_path), 2))

# Output
print("\nOptimal Waypoints Found:")

for point in best_path:
    print("X =", round(point[0], 2),
          "Y =", round(point[1], 2))

print("Total Distance:", round(fitness(best_path), 2))
```

```
Enter start point: 0 0
Enter target point: 10 10
Enter number of nests: 20
Enter maximum rounds: 60
Iteration: 10 Best distance: 14.4
Iteration: 20 Best distance: 14.34
Iteration: 30 Best distance: 14.34
Iteration: 40 Best distance: 14.34
Iteration: 50 Best distance: 14.34
Iteration: 60 Best distance: 14.34

Optimal Waypoints Found:
X = 1.78 Y = 0.98
X = 4.52 Y = 3.21
X = 5.58 Y = 5.18
Total Distance: 14.34
```